

Practical Lab7: DNS Introduction and Traffic Analysis

Student name: Afeez Bolaji Usman

Student ID: 202514326WY

Course Code: SBT-DF203

Course Title : Basic Networking Skills for Digital Forensics

Lab Title: Lab 7 - DNS Introduction and Traffic Analysis

Instructor Name: Aminu Idris

Date: 22/9/2026

Executive Summary

This report documents a normal DNS forensic baseline exercise performed on an isolated Kali Linux VM, per SBT-DF203 Lab 7. The configured local resolver was identified at two layers (systemd-resolved stub 127.0.0.53, forwarding to upstream resolver 192.168.159.2). Controlled dig queries were run against example.com (A, AAAA, MX, NS) to establish baseline record-type behaviour, followed by fresh packet captures of individual A and NS queries to extract and verify query/response fields, transaction IDs, response codes, answers and TTLs at the packet level.

Three independent DNS transactions were matched end-to-end using transaction ID plus endpoint-tuple verification (one drawn from the supplied training evidence, dig_dns.pcap, and two freshly captured). A real browser DNS burst was captured during a live page load of www.bbc.com, revealing 26 distinct queried domains across main-site, CDN, analytics, advertising and consent-management categories. A DNS answer (www.bbc.com to 146.75.76.81) was directly correlated with a TCP SYN to that same address 3.4ms later, confirming the causal link between resolution and connection. Finally, DNS evidence was recovered from a previously captured SMTP session, showing the client resolving mail.patriots.in to 74.53.140.153 just 34ms before the SMTP session began, corroborating the resolution-to-connection pattern observed elsewhere in this report.

Resolver and Environment

Layer	Resolver	Role
Stub resolver (local)	127.0.0.53	systemd-resolved's local stub — all applications on this VM query this address first; traffic never leaves loopback
Upstream resolver (actual)	192.168.159.2	The real DNS server queries are forwarded to; this is the address visible in any packet capture on eth0, and coincides with this VM's default gateway

This two-layer architecture explains an important capture consideration observed throughout this lab: dig itself reports SERVER: 127.0.0.53, but any packet capture taken on eth0 shows queries destined for 192.168.159.2 instead - the stub-to-upstream hop is invisible to on-the-wire capture. This matches the manual's own Interface Note about queries appearing on lo before a physical interface.

Evidence Integrity

Item	Value
Supplied evidence file	evidence/dig_dns.pcap (ICDFA-authorized training resource)
Original SHA-256	9a7c1b95aa6d60f8ac0ee9a79e06ae51145bb7768974526546316cd3dc91375d
Working-copy SHA-256	9a7c1b95aa6d60f8ac0ee9a79e06ae51145bb7768974526546316cd3dc91375d (matches original)
Fresh capture 1 (A query)	evidence/fresh_dig_dns.pcapng — SHA-256 7d4801d9c63acc9b14eb598255bae00c66f85896fe207de7cb972e30af307a0a
Fresh capture 2 (NS query)	evidence/fresh_dig_dns2.pcapng
Browser DNS capture	evidence/browser_dns.pcapng and evidence/dns_and_tcp.pcapng

Chain-of-Custody Worksheet

Field	Entry
Case/lab identifier	SBT-DF203-Lab7-Afeez Bolaji Usman
Trainee name	Afeez Bolaji Usman
Student ID	202514326WY
Date and time started	22/9/2026
Evidence file name(s)	dig_dns.pcap, fresh_dig_dns.pcapng, fresh_dig_dns2.pcapng, browser_dns.pcapng, dns_and_tcp.pcapng
Source or generation method	dig_dns.pcap: downloaded from the ICDDFA-authorized training resource. All other captures: self-generated on the Kali VM using tshark, per manual Parts B/E/F
Analysis workstation/VM	Kali Linux (kali@kali)
Notes on any changes	No original evidence modified; all analysis performed on working copies. First attempts at both the fresh-query capture (AppArmor/sudo timing) and the browser capture (missed navigation window, then a DNS-only filter that excluded subsequent connections) were retried and documented as limitations rather than discarded silently — see Section 9

```
(kali㉿kali)-[~]
$ mkdir -p ~/SBT-DF203-Lab7/{evidence,working,exported,reports,screenshots,scripts}

(kali㉿kali)-[~]
$ cd ~/SBT-DF203-Lab7

(kali㉿kali)-[~/SBT-DF203-Lab7]
$ pwd
/home/kali/SBT-DF203-Lab7

(kali㉿kali)-[~/SBT-DF203-Lab7]
$ find . -maxdepth 1 -type d -print
.
./exported
./evidence
./working
./screenshots
./scripts
./reports
```

```

(kali㉿kali)-[~]
$ cd SBT-DF203-Lab7

(kali㉿kali)-[~/SBT-DF203-Lab7]
$ cp --preserve=timestamps evidence/dig_dns.pcap working/dig_dns_working.pcap

(kali㉿kali)-[~/SBT-DF203-Lab7]
$ sha256sum evidence/dig_dns.pcap working/dig_dns_working.pcap | tee reports/dns_capture_hashes.txt
9a7c1b95aa6d60f8ac0ee9a79e06ae51145bb7768974526546316cd3dc91375d  evidence/dig_dns.pcap
9a7c1b95aa6d60f8ac0ee9a79e06ae51145bb7768974526546316cd3dc91375d  working/dig_dns_working.pcap

(kali㉿kali)-[~/SBT-DF203-Lab7]
$ █

```

```

(kali㉿kali)-[~/SBT-DF203-Lab7]
$ cat /etc/resolv.conf | tee reports/resolv_conf.txt
resolvectl status 2>/dev/null | tee reports/resolvectl_status.txt || true
# This is /run/systemd/resolve/stub-resolv.conf managed by man:systemd-resolved(8).
# Do not edit.
#
# This file might be symlinked as /etc/resolv.conf. If you're looking at
# /etc/resolv.conf and seeing this text, you have followed the symlink.
#
# This is a dynamic resolv.conf file for connecting local clients to the
# internal DNS stub resolver of systemd-resolved. This file lists all
# configured search domains.
#
# Run "resolvectl status" to see details about the uplink DNS servers
# currently in use.
#
# Third party programs should typically not access this file directly, but only
# through the symlink at /etc/resolv.conf. To manage man:resolv.conf(5) in a
# different way, replace this symlink by a static file or a different symlink.
#
# See man:systemd-resolved.service(8) for details about the supported modes of
# operation for /etc/resolv.conf.

nameserver 127.0.0.53
options edns0 trust-ad
search .
Global
    Protocols: +LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
resolv.conf mode: stub

Link 2 (eth0)
    Current Scopes: DNS LLMNR/IPv4 LLMNR/IPv6
    Protocols: +DefaultRoute +LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
    Current DNS Server: 192.168.159.2
    DNS Servers: 192.168.159.2
    DNS Domain: ~.
    Default Route: yes

Link 3 (docker0)
    Current Scopes: none
    Protocols: -DefaultRoute +LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
    Default Route: no

```

Part A - Query DNS Records with dig

```

(kali@kali) [~/SBT-DF203-Lab7]
$ dig example.com A | tee reports/dig_example_A.txt
dig example.com AAAA | tee reports/dig_example_AAAA.txt
dig example.com MX | tee reports/dig_example_MX.txt
dig example.com NS | tee reports/dig_example_NS.txt
dig +short example.com A | tee reports/dig_example_short.txt

; <<>> DiG 9.20.27-2-Debian <<>> example.com A
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 23517
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags;; udp: 65494
;; QUESTION SECTION:
;example.com.                IN      A
;; ANSWER SECTION:
example.com.                5       IN      A      104.20.23.154
example.com.                5       IN      A      172.66.147.243

;; Query time: 199 msec
;; SERVER: 127.0.0.53#53(127.0.0.53) (UDP)
;; WHEN: Tue Sep 22 06:14:54 EDT 2026
;; MSG SIZE rcvd: 72

; <<>> DiG 9.20.27-2-Debian <<>> example.com AAAA
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 36407
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags;; udp: 65494
;; QUESTION SECTION:
;example.com.                IN      AAAA
;; ANSWER SECTION:
example.com.                5       IN      AAAA
;; Query time: 207 msec
;; SERVER: 127.0.0.53#53(127.0.0.53) (UDP)
;; WHEN: Tue Sep 22 06:14:54 EDT 2026
;; MSG SIZE rcvd: 40

```

```

; <<>> DiG 9.20.27-2-Debian <<>> example.com MX
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 5562
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags;; udp: 65494
;; QUESTION SECTION:
;example.com.                IN      MX
;; ANSWER SECTION:
example.com.                5       IN      MX      0 .

;; Query time: 235 msec
;; SERVER: 127.0.0.53#53(127.0.0.53) (UDP)
;; WHEN: Tue Sep 22 06:14:54 EDT 2026
;; MSG SIZE rcvd: 55

; <<>> DiG 9.20.27-2-Debian <<>> example.com NS
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 45710
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags;; udp: 65494
;; QUESTION SECTION:
;example.com.                IN      NS
;; ANSWER SECTION:
example.com.                5       IN      NS      hera.ns.cloudflare.com.
example.com.                5       IN      NS      elliot.ns.cloudflare.com.

;; Query time: 203 msec
;; SERVER: 127.0.0.53#53(127.0.0.53) (UDP)
;; WHEN: Tue Sep 22 06:14:54 EDT 2026
;; MSG SIZE rcvd: 95
172.66.147.243
104.20.23.154

```

Query	Staus	Answer	Query Time
A	NOERROR, 2 answers	104.20.23.154, 172.66.147.243	199ms
AAAA	NOERROR, 0 answers	none — domain has no IPv6 record configured	207ms
MX	NOERROR, 1 answer	0, — a null MX record (RFC 7505), explicitly saying "this domain accepts no mail"	235ms

NS	NOERROR, 2 answers	hera.ns.cloudflare.com., elliott.ns.cloudflare.com.	203ms
----	--------------------	--	-------

1. SERVER: 127.0.0.53 confirms exactly what we predicted — dig talks to the local stub resolver first, which then forwards upstream to 192.168.159.2 (you won't see that hop in dig's own output, only in the packet capture later).
2. TTL = 5 seconds on every record — unusually short. Real-world domains typically cache for minutes-to-hours; a 5-second TTL is characteristic of Cloudflare's proxied/edge-served domains, which deliberately keep TTLs very low so they can rapidly redirect traffic (failover, load balancing, etc.) without waiting on client caches. Worth flagging as a "normal variation" observation for Part 9 (Limitations and normal variations) rather than an anomaly.
3. A/NS records point to Cloudflare infrastructure (104.20.x.x, 172.66.x.x, *.ns.cloudflare.com) — example.com is now served behind Cloudflare's edge network rather than a single static origin server, which is why you get 2 A answers (anycast/load-distributed) instead of the single fixed IP older references to "example.com" might describe.
4. AAAA = 0 answers, NOERROR (not NXDOMAIN) — this is the correct distinction to document: the domain exists and DNS resolution worked, it simply has no IPv6 address configured. This is a good teaching example of "empty answer ≠ failure."

IN = Internet class (the only class used in practice today)

A = maps a name to an IPv4 address

AAAA = maps a name to an IPv6 address

MX = identifies the mail server(s) responsible for accepting email for the domain (priority + hostname)

NS = identifies the authoritative nameservers for the domain

Part B - Capture a Fresh DNS Query

```
(kali㉿kali)-[~/SBT-DF203-Lab7]
$ sudo -v
IFACE=eth0
sudo tshark -i "$IFACE" -f 'port 53' -a duration:25 -w /tmp/fresh_dig_dns.pcapng 6
sleep 3
dig +noedns example.com A >/dev/null
wait
sudo mv /tmp/fresh_dig_dns.pcapng ~/SBT-DF203-Lab7/evidence/fresh_dig_dns.pcapng
sudo chown kali:kali ~/SBT-DF203-Lab7/evidence/fresh_dig_dns.pcapng
sha256sum evidence/fresh_dig_dns.pcapng | tee reports/fresh_dns_sha256.txt
[2] 50640
Running as user "root" and group "root". This could be dangerous.
Capturing on 'eth0'
2
[2]  done      sudo tshark -i "$IFACE" -f 'port 53' -a duration:25 -w
7d4801d9c63acc9b14eb598255bae00c66f85896fe207de7cb972e30af307a0a  evidence/fresh_dig_dns.pcapng
```

1. The query goes straight to 192.168.159.2 - The real upstream resolver, on the wire - not 127.0.0.53. This is the stub-resolver behavior we predicted: 127.0.0.53 traffic never left the

loopback interface, so it's invisible to a capture on eth0, but the actual forwarded query to my real DNS server is fully visible here.

2. Transaction ID **0x6b54** matches on both the query and response - this is the field that lets a resolver/client match a response to its original query.
3. Response time: 0.203s (203ms) — matches almost exactly what dig itself reported (199-235ms across the five earlier queries), good cross-validation between the client-reported timing and the packet-capture timing.
4. +noedns didn't fully suppress the EDNS OPT record — OPT still appears in both packets. dig's +noedns flag disables sending EDNS query options in some versions, but the OPT pseudo-record can still appear depending on dig/resolver version — not a capture error, just a minor dig-version quirk.

Part C - Extract DNS Query and Response Fields

```
(kali㉿kali)-[~/SBT-DF203-Lab7]
$ PCAP=evidence/fresh_dig_dns.pcapng

tshark -r "$PCAP" -Y 'dns.flags.response==0' -T fields \
-e frame.number -e frame.time -e ip.src -e udp.srcport -e ip.dst -e udp.dstport \
-e dns.id -e dns.qry.name -e dns.qry.type \
| tee reports/dns_queries.tsv

tshark -r "$PCAP" -Y 'dns.flags.response==1' -T fields \
-e frame.number -e frame.time -e ip.src -e udp.srcport -e ip.dst -e udp.dstport \
-e dns.id -e dns.flags.rcode -e dns.count.answers -e dns.a -e dns.aaaa -e dns.resp.ttl \
| tee reports/dns_responses.tsv
1      2026-09-22T06:46:59.964587148-0400      192.168.159.128 58801      192.168.159.2   53      0x
6b54   example.com      1
2      2026-09-22T06:47:00.167664058-0400      192.168.159.2   53      192.168.159.128 58801  0x
6b54   0      2      172.66.147.243,104.20.23.154      5,5
```

Query fields	Value
Frame	1
Timestamp	06:46:59.964587148
Client source	192.168.159.128:58801
Resolver destination	192.168.159.2:53
Transaction	ID0x6b54
Query	nameexample.com
Query	type1 (A)

Response fields	Value
Frame	2
Timestamp	06:47:00.167664058
Resolver source	192.168.159.2:53
Client destination	192.168.159.128:58801
Transaction ID	0x6b54 (matches query ✓)
Response	code 0 (NOERROR)
Answer count	2
A answers	172.66.147.243, 104.20.23.154
TTL5,	5 (seconds — matches the dig output exactly)

Part D - Match Queries to Responses

Transaction ID	Query name	Type	Client/Resolver	Response Code	Answer(s)	TTL	Time Delta
0xb4a3	google.com	A	136.160.215.15 ↔ 192.168.2.31 (supplied evidence, dig_dns.pcap)	0 (NOERROR)	142.251.163.113, .139, .102, .100, .138, .101 (6 answers)	25s	2.8ms
0x6b54	example.com	A	192.168.159.128 ↔ 192.168.159.2 (our fresh capture)	0 (NOERROR)	172.66.147.243, 104.20.23.154 (2 answers)	5s	203.1ms
0x76ba	example.com	NS	192.168.159.128 ↔ 192.168.159.2 (our fresh capture)	0 (NOERROR)	hera.ns.cloudflare.com, elliottns.cloudflare.com (2 answers)	5s	203.0ms

1. Match query and response using transaction ID plus the endpoint tuple.

Transaction	Query endpoint tuple	Response endpoint tuple	Match?
0xb4a	3136.160.215.15:33039 → 192.168.2.31:53	192.168.2.31:53 → 136.160.215.15:33039	Ports and IPs mirror exactly
0x6b54	192.168.159.128:58801 → 192.168.159.2:53	192.168.159.2:53 → 192.168.159.128:58801	Ports and IPs mirror exactly
0x76ba	192.168.159.128 → 192.168.159.2:53	192.168.159.2:53 → 192.168.159.128	Same client/resolver pair as 0x6b54, distinct transaction ID

2. Confirm that the response source is the queried resolver

0xb4a3: queried 192.168.2.31, response came from 192.168.2.31

0x6b54: queried 192.168.159.2, response came from 192.168.159.2

0x76ba: queried 192.168.159.2, response came from 192.168.159.2

3. Response time (calculated from frame timestamps, not dig's self-reported time — an independent cross-check)

Transaction	Query timestamp	Response timestamp	Delta
0xb4a3	10:01:02.172039301	10:01:02.174822587	0.002783286s (2.78ms)
0x6b54	06:46:59.964587148	06:47:00.167664058	0.203076910s (203.08ms)
0x76ba	(frame 1, capture-relative 0.000000000)(frame 2, capture-relative		0.203031059)
			0.203031059s (203.03ms)

4. CNAME chains and multiple A/AAAA answers

No CNAME chains observed in any of the three transactions — every query resolved directly to A or NS records without an intermediate alias hop. Worth noting as a "not observed" finding rather than skipping it silently, since the manual explicitly asks you to check.

Multiple answers observed in two of three: google.com A returned 6 A records (anycast load distribution), example.com A returned 2 A records (Cloudflare edge pool). The example.com NS query also returned 2 NS records, though that's expected/normal for NS (domains typically have 2+ authoritative nameservers for redundancy, not the same "multiple answer" phenomenon as A-record load balancing).

Part E - Analyze DNS Generated by a Browser

```

(kali㉿kali)-[~/SBT-DF203-Lab7]
$ sudo -v
IFACE=eth0
sudo tshark -i "$IFACE" -f 'port 53' -a duration:35 -w /tmp/browser_dns2.pcapng &
sleep 5
echo "CAPTURE RUNNING — go to your browser NOW and type a URL, then press Enter"
wait
sudo mv /tmp/browser_dns2.pcapng ~/SBT-DF203-Lab7/evidence/browser_dns.pcapng
sudo chown kali:kali ~/SBT-DF203-Lab7/evidence/browser_dns.pcapng
[2] 90313
Running as user "root" and group "root". This could be dangerous.
Capturing on 'eth0'
CAPTURE RUNNING — go to your browser NOW and type a URL, then press Enter
204
[2] done sudo tshark -i "$IFACE" -f 'port 53' -a duration:35 -w

(kali㉿kali)-[~/SBT-DF203-Lab7]
$ tshark -r evidence/browser_dns.pcapng -Y 'dns.flags.response==0' -T fields -e dns.qry.name -e dns.qry.type \
| sort | uniq -c | sort -nr | tee reports/browser_dns_inventory.txt
  8 a1.api.bbc.com 28
  8 a1.api.bbc.com 1
  6 browser-intake-datadoghq.com 28
  6 browser-intake-datadoghq.com 1
  4 uk-script.dotmetrics.net 28
  4 uk-script.dotmetrics.net 1
  4 static.chartbeat.com 28
  4 static.chartbeat.com 1
  4 pagead2.googlesyndication.com 28
  4 pagead2.googlesyndication.com 1
  4 logx.optimizely.com 28

```

```

  1 www.bbc.co.uk 28
  1 www.bbc.co.uk 1
  1 www.bbc.com 65
  1 www.bbc.com 28
  1 www.bbc.com 1
  1 web-cdn.api.bbc.co.uk 28
  1 web-cdn.api.bbc.co.uk 1
  1 static-web-assets.gnl-common.bbcverticals.com 28
  1 static.files.bbc.co.uk 28
  1 static.files.bbc.co.uk 1
  1 static.bbc.co.uk 28
  1 static.bbc.co.uk 1
  1 mybbc-analytics.files.bbc.co.uk 28
  1 mybbc-analytics.files.bbc.co.uk 1
  1 ichef.bbc.co.uk 28
  1 ichef.bbc.co.uk 1
  1 gn-web-assets.api.bbc.com 28
  1 gn-web-assets.api.bbc.com 1
  1 emp.bbc.co.uk 28
  1 emp.bbc.co.uk 1
  1 cdn.tinypass.com 28
  1 cdn.tinypass.com 1
  1 cdn.optimizely.com 28
  1 cdn.optimizely.com 1
  1 cdnjs.cloudflare.com 28
  1 cdnjs.cloudflare.com 1
  1 c2-eu.piano.io 28
  1 c2-eu.piano.io 1
  1 buy-eu.piano.io 28
  1 buy-eu.piano.io 1
  1 a4621041136.cdn.optimizely.com 28
  1 a4621041136.cdn.optimizely.com 1

```

Type 28 = AAAA, type 1 = A, type 65 = HTTPS (the newer HTTPS/SVCB resource record type, used for connection-parameter hints like ALPN — a good "normal variation" to note, since the manual doesn't explicitly cover this record type but it's increasingly common in real traffic).

Part F - Correlate DNS with Subsequent Connections

```
(kali@kali)-[~/SBT-DF203-Lab7]
$ tshark -r evidence/browser_dns.pcapng -Y 'dns.a' -T fields -e frame.time_epoch -e dns.qry.name -e dns.a \
| tee reports/dns_A_answers.tsv

tshark -r evidence/browser_dns.pcapng -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields \
-e frame.time_epoch -e ip.dst -e tcp.dstport \
| tee reports/subsequent_tcp_destinations.tsv
1790078770.602470588 www.bbc.com 146.75.76.81
1790078772.416534590 static.bbci.co.uk 23.14.140.171
1790078772.416534743 cdn.tinypass.com 104.18.160.126,104.18.239.248,104.18.223.248,104.18.144.126,104.18.176.126
1790078772.416535008 emp.bbci.co.uk 23.14.140.171
1790078772.416535051 static.files.bbci.co.uk 23.14.140.171
1790078772.617009121 gn-web-assets.api.bbc.com 18.172.134.125,18.172.134.80,18.172.134.88,18.172.134.127
1790078772.825691302 ichef.bbci.co.uk 23.14.140.171
1790078774.663257715 www.bbc.co.uk 146.75.76.81
1790078776.753568975 cdn.optimizely.com 104.18.65.57,104.18.66.57
1790078776.949802396 web-cdn.api.bbci.co.uk 13.227.87.70,13.227.87.2,13.227.87.40,13.227.87.49
1790078777.828308009 c2-eu.piano.io 104.16.144.111,104.16.143.111
1790078782.080330415 mybbc-analytics.files.bbci.co.uk 13.225.47.95,13.225.47.67,13.225.47.88,13.225.47.51
1790078782.085030809 a4621041136.cdn.optimizely.com 104.18.66.57,104.18.65.57
1790078782.091935202 buy-eu.piano.io 104.16.143.111,104.16.144.111
1790078782.767268017 cdnjs.cloudflare.com 104.17.24.14,104.17.25.14
```

```
(kali@kali)-[~/SBT-DF203-Lab7]
$ tshark -r evidence/dns_and_tcp.pcapng -Y 'dns.a' -T fields -e frame.time_epoch -e dns.qry.name -e dns.a \
| tee reports/dns_A_answers2.tsv
tshark -r evidence/dns_and_tcp.pcapng -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields -e frame.time_epoch -e ip.dst -e tcp.dstport \
| tee reports/subsequent_tcp_destinations2.tsv
1790079700.367525592 www.bbc.com 146.75.76.81
1790079700.408774752 static.files.bbci.co.uk 23.14.140.171
1790079700.408774832 ichef.bbci.co.uk 23.14.140.171
1790079701.764714138 dotcom.bbc-reporting-api.app 34.111.72.164
1790079701.830117552 static.bbci.co.uk 23.14.140.171
1790079701.835624787 cdn.tinypass.com 104.18.239.248,104.18.160.126,104.18.223.248,104.18.144.126,104.18.176.126
1790079701.846379496 emp.bbci.co.uk 23.14.140.171
1790079702.041936103 gn-web-assets.api.bbc.com 18.172.134.88,18.172.134.80,18.172.134.127,18.172.134.125
1790079704.251660446 www.bbc.co.uk 146.75.76.81
1790079704.325879083 a4621041136.cdn.optimizely.com 104.18.66.57,104.18.65.57
1790079707.750640145 c2-eu.piano.io 104.16.144.111,104.16.143.111
1790079707.754648362 mybbc-analytics.files.bbci.co.uk 13.225.47.51,13.225.47.88,13.225.47.95,13.225.47.67
1790079709.836083044 buy-eu.piano.io 104.16.143.111,104.16.144.111
1790079709.840668842 cdnjs.cloudflare.com 104.17.24.14,104.17.25.14
1790079700.370933864 146.75.76.81 443
1790079700.411651418 23.14.140.171 443
1790079700.412219989 23.14.140.171 443
1790079701.648703746 23.14.140.171 443
1790079701.766101254 34.111.72.164 443
1790079701.831624314 23.14.140.171 443
1790079701.837952684 104.18.239.248 443
1790079701.848574091 23.14.140.171 443
1790079701.913457660 34.111.72.164 443
1790079702.055083092 18.172.134.88 443
1790079702.264420367 23.14.140.171 443
1790079702.759946957 23.14.140.171 443
1790079702.808808704 23.14.140.171 443
1790079702.937646112 23.14.140.171 443
1790079704.256111785 146.75.76.81 443
1790079704.328189473 104.18.66.57 443
1790079707.752233609 104.16.144.111 443
1790079707.757364764 13.225.47.51 443
1790079708.011318278 13.225.47.51 443
1790079708.024491506 13.225.47.51 443
1790079709.842282246 104.16.143.111 443
1790079709.843241311 104.17.24.14 443
1790079710.098560207 104.17.25.14 443
1790079710.384249933 23.14.140.171 443
1790079710.629601133 23.14.140.171 443
```

Method: An initial capture (browser_dns.pcapng) was filtered to port 53 only, which excluded all subsequent TCP connections by design. This scope limitation was documented, then a second, broader capture (dns_and_tcp.pcapng, filter port 53 or tcp port 443) was taken during a repeat page load of the same site, capturing both DNS resolution and the resulting TLS connections together.

Event	Timestamp (epoch)	Delta
DNS response: www.bbc.com A → 146.75.76.81	1790079700.367525592	-----
TCP SYN → 146.75.76.81:443	1790079700.370933864	+0.0034s (3.4ms)

The client initiated a TCP connection to the exact DNS-returned IP just 3.4ms after the response — a direct, unambiguous correlation.

Corroborating example: www.bbc.co.uk resolved to the same IP (146.75.76.81) and was followed by a SYN to that address 4.5ms later, consistent with both hostnames being served from the same edge endpoint via SNI-based virtual hosting.

Explaining an apparent mismatch: cdn.tinypass.com resolved to five A records (104.18.239.248, 104.18.160.126, 104.18.223.248, 104.18.144.126, 104.18.176.126), but only one (104.18.239.248) was actually used for the subsequent connection. This is normal client/OS behaviour when a domain is served by a load-balanced or CDN-fronted pool: multiple valid answers are returned, and the client selects one — the unused addresses are not evidence of an error.

Conclusion: DNS resolution was directly and verifiably linked to subsequent connection behaviour for this capture. The initial filtered capture's inability to show this same correlation was itself an instructive finding: a capture scoped only to port 53 cannot demonstrate downstream connection activity, since that traffic is excluded before it is ever written to disk — absence of evidence in a narrowly filtered capture does not indicate absence of the underlying connection.

Part G - DNS Analysis Within SMTP Evidence

```
(kali㉿kali)-[~/SBT-DF203-Lab7]
$ tshark -r ../SBT-DF203-Lab4/working/smtp_working.pcap -Y 'dns' -T fields \
-e frame.number -e frame.time -e dns.qry.name -e dns.qry.type -e dns.a -e dns.resp.name -e dns.res
p.ttl \
| tee reports/smtp_dns_correlation.tsv
1      2009-10-05T02:06:07.492060000-0400      mail.patriots.in      1
2      2009-10-05T02:06:07.526085000-0400      mail.patriots.in      1      74.53.140.153      ma
il.patriots.in,patriots.in,patriots.in,patriots.in      10827,10828,82828,82828
(kali㉿kali)-[~/SBT-DF203-Lab7]
$
```

Field	Value
Query name	mail.patriots.in
Query type	1 (A)
Query time	2009-10-05 02:06:07.492060

Response time	2009-10-05 02:06:07.526085
Response delta	0.034025s (34ms)
Answer	74.53.140.153 — matches exactly the server IP the SMTP session subsequently connects to
Response names (CNAME-style chain in the resp.name field)	mail.patriots.in, patriots.in, patriots.in, patriots.in
TTLs	10827, 10828, 82828, 82828 (seconds — roughly 3 hours and 23 hours respectively, much longer-lived than the 5s Cloudflare TTLs we saw in Parts A–D, consistent with a more traditionally-cached mail record rather than an edge/anycast service)

Forensic correlation: the client queried mail.patriots.in, received 74.53.140.153 as the answer, and — as we established in Lab 4 — proceeded to open the SMTP session directly to 74.53.140.153:25. This is a textbook DNS→connection correlation, and it directly confirms the mail server's identity was resolved via DNS immediately before use, exactly as the manual's Part G is designed to demonstrate.

9. Limitations and Normal Variations

Normal variations observed and documented throughout this lab:

- Empty answer set with NOERROR status (example.com AAAA) — correctly distinguished from NXDOMAIN; the domain exists, it simply has no record of that type.
- Null MX record (RFC 7505) explicitly declaring a domain accepts no mail, rather than omitting the MX record entirely.
- Unusually short TTLs (5s) on a Cloudflare-edge-served domain, versus much longer TTLs (hours) on a traditionally-hosted mail domain (Lab 4 evidence) — TTL length reflects infrastructure and caching strategy, not correctness.
- Multiple A answers for load-balanced/anycast services (google.com: 6 answers; various BBC CDN domains: 4-5 answers each) — a client using only one of several valid answers is expected, not a mismatch.
- Two distinct hostnames (www.bbc.com, www.bbc.co.uk) resolving to the identical IP via SNI-based virtual hosting.
- HTTPS/SVCB (type 65) records appearing alongside classic A/AAAA queries in modern browser traffic.

Limitations encountered:

- A capture filter scoped only to DNS traffic (port 53) cannot demonstrate subsequent connection behaviour, since non-matching packets are excluded before being written to disk. This was directly demonstrated in Part E/F and required a second, deliberately broader capture (port 53 or tcp port 443) to resolve.
- A capture's evidentiary value is bounded by its start time and scope: the Lab 4 SMTP capture was originally inventoried as SMTP-only traffic, but was found on closer inspection (via this lab's DNS-specific filtering) to include two earlier DNS frames not surfaced by the original inventory — a reminder to re-examine full frame ranges with protocol-specific filters even after an initial pass.
- Browser-generated background noise (Firefox's ads.mozilla.org canary queries on private-window

startup) can be mistaken for real page-load traffic if a capture window does not align precisely with actual user navigation; this was encountered directly in Part E's first attempt and required a retry with better-timed navigation.

- dig's +noedns flag did not fully suppress the EDNS OPT pseudo-record in the resulting packet capture
- a minor dig/resolver-version quirk, not a capture error

Appendix

Findings Worksheet

Field	Finding
Configured resolver IP	Stub: 127.0.0.53 Upstream/actual: 192.168.159.2
Client source port	58801 (example.com A query, fresh capture)
Resolver destination port	53
Transaction ID	0x6b54 (example.com A); 0x76ba (example.com NS); 0xb4a3 (google.com A, supplied evidence)
Query name and type	example.com A/AAAA/MX/NS (dig); example.com A/NS (packet capture); google.com A (supplied evidence); mail.patriots.in A (Lab 4 SMTP evidence)
Response code	0 / NOERROR in every transaction examined
Answer IP(s)	example.com: 104.20.23.154, 172.66.147.243; google.com: 6 records in the 142.251.163.x range; mail.patriots.in: 74.53.140.153
TTL	5s (example.com, Cloudflare-edge); 25s (google.com); 10827-82828s (mail.patriots.in)
Query-response time delta	2.8ms (supplied evidence, same-segment resolver); ~203ms (this VM's fresh captures); 34ms (Lab 4 SMTP evidence)
Subsequent connection correlation	Confirmed: www.bbc.com DNS answer 146.75.76.81 followed by TCP SYN to the same address 3.4ms later; mail.patriots.in DNS answer 74.53.140.153 followed by SMTP session to the same address 34ms later (Lab 4 evidence)
Normal variations observed	Empty-answer NOERROR, null MX, short vs long TTL contrast, multiple A answers, shared-IP virtual hosting, HTTPS/SVCB record type